

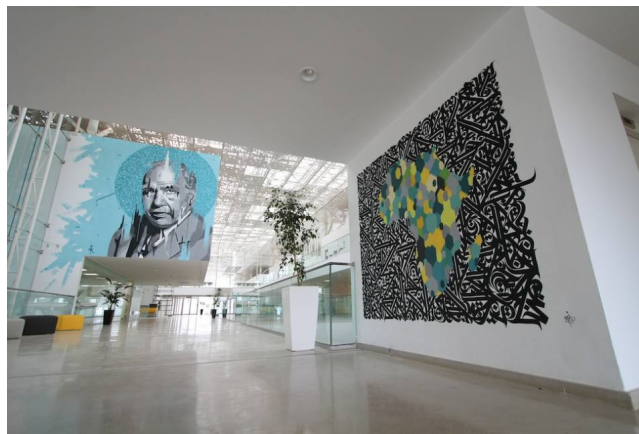


ÉCOLE CENTRALE CASABLANCA

PROJET LEARNING BY DOING

Aqua-Cleaner

Robot Autonome de Collecte de Déchets Flottants



Réalisé par :

EL ALLAoui Mohamed El
Mehdi
CHAKIR Mohamed
Chekaba Yassine
HASSAR Nada
VIANNEY Léandre

Encadrant :

Mr. DAMIL Noureddine

Juin 2026

Table des matières

1	Remerciements	3
2	Résumé	4
3	Introduction	5
4	Développement du projet	6
4.1	Le contexte et problématique	6
4.2	État de l’art	6
4.2.1	Solutions manuelles et passives	7
4.2.2	Embarcations motorisées et robots semi-autonomes	7
4.3	Méthodologie et mise en place de la solution	7
4.4	Choix technologiques et architecture matérielle	8
4.5	Planification	9
4.5.1	Organisation de l’équipe	9
4.5.2	Méthodologie de conduite de projet	10
4.6	Réalisation et tests	10
4.6.1	Réalisation de la structure mécanique	10
4.6.2	Conception et impression des flotteurs	10
4.6.3	Conception du boîtier électronique	12
4.6.4	Présentation détaillée des composants électroniques et mécaniques	13
4.6.5	Stratégie de priorisation par distance euclidienne	25
4.6.6	Régulation de la trajectoire par correction d’erreur	25
4.6.7	Propulsion différentielle	25
4.7	Les résultats obtenus suivis des commentaires et d’analyses	26
4.7.1	Détection des déchets par intelligence artificielle (YOLOv8 Nano)	26
4.7.2	Test de l’algorithme de détection et de priorisation par distance	28
4.7.3	Application utilisateur : Analyse des déchets collectés et estimation de la source et controle manuelle du robot	30
5	Conclusion générale	31
6	Références	31
7	Annexes	31

1 Remerciements

Nous tenons à exprimer notre sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nous remercions en premier lieu notre encadrant Monsieur DAMIL pour son soutien constant, ses conseils avisés et sa disponibilité tout au long de cette expérience d'apprentissage par la pratique. Sa guidance nous a permis de structurer notre démarche et d'affronter les défis techniques avec méthode.

Nos remerciements vont également à l'équipe pédagogique de l'École Centrale Casablanca, qui a conçu le programme *Learning by Doing*, offrant aux étudiants l'opportunité de mener des projets d'ingénierie concrets. Cette approche pédagogique favorise l'acquisition de compétences transversales essentielles, alliant rigueur scientifique, créativité et travail en équipe.

Nous remercions les responsables du FabLab de l'école pour leur accueil et l'accès aux équipements de fabrication numérique, sans lesquels le prototypage de notre système n'aurait pas été possible.

Enfin, nous remercions nos familles et proches pour leur soutien moral tout au long de cette période intensive de travail.

2 Résumé

Le projet **Aqua-Cleaner** vise à concevoir un robot catamaran autonome dédié à la collecte des macro-déchets flottants dans les ports et marinas.

Face à l'accumulation critique de débris plastiques dans ces zones semi-fermées, la méthodologie associe une structure bi-coque stable en PLA haute densité à un système intelligent de vision par ordinateur. L'unité centrale Raspberry Pi 4 capture le flux vidéo et délègue l'inférence en temps réel à un PC distant exécutant le modèle YOLOv8 Nano. La trajectoire est ajustée par propulsion différentielle via un algorithme de distance euclidienne et un correcteur PID.

Les tests valident cette architecture distribuée, atteignant une détection fluide à 32 FPS en navigation autonome. Une application web dédiée assure le suivi, le comptage et l'analyse statistique de la pollution traitée.

Ce travail démontre la viabilité d'une solution robotique bas coût et durable, ouvrant la perspective d'une automatisation complète de la dépollution portuaire.

3 Introduction

La pollution marine par les déchets constitue l'un des enjeux environnementaux les plus préoccupants du XXI^e siècle. À l'échelle mondiale, on estime que plus de 8 millions de tonnes de plastique rejoignent les océans chaque année, selon les données de l'UNEP. En Afrique subsaharienne et au Maghreb, la situation est particulièrement critique : la croissance urbaine rapide, combinée à des systèmes de gestion des déchets encore en développement, entraîne une accumulation significative de déchets dans les zones côtières, les ports et les marinas.

Les ports et marinas sont des zones d'accumulation privilégiées. Leur configuration semi-fermée, la faible circulation des eaux et la concentration d'activités maritimes et humaines créent des conditions propices à l'accumulation de macro-déchets flottants. Ces détritiques nuisent non seulement à l'écosystème marin local, mais représentent aussi un risque pour la navigation, dégradent l'attractivité touristique et génèrent des coûts de nettoyage importants pour les autorités portuaires.

Face à ce constat, le projet **Aqua-Cleaner** naît d'une volonté de proposer une réponse technologique innovante, autonome et écologique à ce problème. Il s'inscrit pleinement dans les objectifs de développement durable (ODD 14 – Vie aquatique) et dans la dynamique d'innovation technologique que cherche à insuffler l'École Centrale Casablanca à travers son programme *Learning by Doing*. L'objectif est de concevoir un robot autonome de type catamaran capable de détecter, cibler et collecter ces déchets de manière intelligente.

Pour présenter le déroulement et les résultats de ce travail, ce rapport est structuré en plusieurs parties. Après un résumé global du projet, nous exposerons dans un premier temps le contexte général, la problématique ainsi que la planification temporelle de nos objectifs. Dans un deuxième temps, un état de l'art dressera un panorama des concepts technologiques existants et justifiera les choix techniques retenus pour notre système. Ensuite, nous détaillerons la mise en place de notre solution ainsi que la réalisation concrète du prototype, tant sur le plan de la structure mécanique que de l'intégration des composants électroniques. Enfin, nous analyserons les résultats obtenus lors des phases de tests, notamment les performances de l'algorithme de détection par intelligence artificielle et le système de priorisation des cibles, avant de conclure sur les perspectives d'amélioration de notre robot.

4 Développement du projet

4.1 Le contexte et problématique

Origine et zones d'accumulation des déchets [1]

Les déchets présents en milieu portuaire proviennent de sources multiples, principalement liées aux activités humaines terrestres et côtières. Environ 80% des déchets marins sont d'origine terrestre : ils résultent des rejets urbains et industriels, de la mauvaise gestion des déchets, ainsi que des activités touristiques, avant d'être transportés vers le milieu marin par les cours d'eau, les réseaux de drainage et le ruissellement.

TABLE 1 – Sources et proportions estimées des déchets portuaires

Source	Proportion (%)	Types de déchets dominants
Rejets urbains et fluviaux	45	Plastiques, papiers, emballages
Activités portuaires	25	Cordages, bâches, hydrocarbures
Navigation de plaisance	15	Bouteilles, canettes, matériaux flottants
Activités touristiques	10	Emballages, déchets alimentaires
Autres (pêche, industrie)	5	Filets, matériaux industriels

Problématique principale

Comment réduire efficacement la pollution causée par les déchets flottants dans les ports et marinas à l'aide d'une solution robotisée autonome, durable et adaptée au contexte africain ?

Cette problématique centrale se décline en plusieurs sous-questions d'ingénierie :

1. Comment **détecter et identifier automatiquement** les déchets flottants dans un environnement marin dynamique avec des conditions lumineuses variables ?
2. Comment **naviguer de manière autonome** en évitant les obstacles et en optimisant les trajectoires de collecte ?
3. Comment **collecter efficacement** différents types et tailles de déchets flottants ?
4. Comment **maximiser l'autonomie énergétique** pour des missions continues prolongées ?
5. Comment **assurer la durabilité et la maintenance** du système dans un environnement marin corrosif ?
6. Comment **maintenir un coût de production** compatible avec un déploiement dans des ports de taille modeste ?

4.2 État de l'art

Cette partie présente les aspects théoriques nécessaires à la compréhension du problème et de la solution. Il peut contenir aussi une description des solutions existantes.

4.2.1 Solutions manuelles et passives

Le nettoyage traditionnel des ports repose sur des interventions manuelles (époussettes) ou des dispositifs passifs et statiques comme les poubelles flottantes de type *Seabin* [3]. Bien que simples, ces méthodes ont un impact très limité : elles demandent un effort humain constant ou restent prisonnières d'une zone fixe, incapables d'aller chercher les déchets là où ils s'accumulent.

4.2.2 Embarcations motorisées et robots semi-autonomes

Pour couvrir plus de surface, des bateaux nettoyeurs habités ou des solutions plus récentes comme le *Jellyfishbot* [2] sont utilisés. Cependant, ces systèmes partagent une dépendance majeure : ils requièrent une présence humaine ou une téléopération constante pour naviguer. De plus, aucun de ces dispositifs n'embarque de système d'analyse intelligente pour cartographier ou comprendre la pollution traitée.

TABLE 2 – Comparatif des solutions existantes avec le positionnement Aqua-Cleaner

Solution	Mode de navigation	Détection intelligente	Analyse des données
Nettoyage manuel	Humain (quai/bateau)	Non	Non
Poubelle flottante (Seabin)	Statique (fixe)	Non	Non
Robot type Jellyfishbot	Téléopéré / Semi-autonome	Non	Non
Aqua-Cleaner	Totalement autonome	Oui (Vision)	Oui (Traçabilité)

4.3 Méthodologie et mise en place de la solution

Démarche de sélection des solutions

La sélection des solutions techniques a été réalisée en quatre étapes structurées :

1. **Brainstorming initial** : Génération d'idées libres sans contraintes a priori. Cette phase a produit plus de 30 concepts différents couvrant toutes les fonctions du robot.
2. **Analyse multicritère** : Évaluation de chaque concept sur 10 critères pondérés (stabilité, traîne hydrodynamique, coût, fabricabilité au FabLab, maintenabilité, écologie, fiabilité, précision de collecte, autonomie, poids).
3. **Prototypage virtuel** : Simulation numérique des solutions les plus prometteuses pour valider les hypothèses de dimensionnement.
4. **Vérification de faisabilité** : Contrôle de la compatibilité avec les contraintes du FabLab (équipements disponibles) et du budget alloué.

Mise en place de la solution :

Solution rejetée 1 : Bras robotisé pour collecte

- **Principe** : Bras articulé à 3 ou 6 degrés de liberté, équipé d'une pince ou d'un panier.
- **Avantages** : Précision de collecte élevée, sélection possible des déchets.
- **Inconvénients** : Complexité mécanique excessive, coût prohibitif, vulnérabilité au milieu marin, instabilité due aux moments de force.

- **Décision : Rejetée** – trop complexe pour un prototype académique et inadaptée à la collecte continue.

Solution rejetée 2 : Système à convoyeur

- **Principe** : Bande transporteuse en caoutchouc immergée partiellement pour remonter les déchets.
- **Avantages** : Collecte continue, bon débit volumique.
- **Inconvénients** : Encombrement important, risques de blocage par les algues et les déchets volumineux, entretien fréquent.
- **Décision : Rejetée** – problèmes d'encrassement prévisibles et faible robustesse.

Solution rejetée 3 : Propulsion hybride diesel-électrique

- **Principe** : Moteur thermique diesel couplé à un générateur pour alimenter les moteurs électriques.
- **Avantages** : Autonomie très étendue, recharge embarquée.
- **Inconvénients** : Émissions de CO₂ et de particules, bruit nuisible à la faune marine, complexité de maintenance, risque de fuite d'hydrocarbures.
- **Décision : Rejetée** – incompatible avec les objectifs écologiques fondamentaux du projet.

Solution rejetée 4 : Navigation par câble guidé

- **Principe** : Robot guidé par un câble le long de lignes prédéfinies dans le port.
- **Avantages** : Fiabilité élevée, pas de problème de communication sans fil.
- **Inconvénients** : Zone couverte très limitée, risque d'enchevêtrement avec les embarcations, infrastructure fixe coûteuse.
- **Décision : Rejetée** – pas adaptable et contraire à l'objectif d'autonomie complète.

4.4 Choix technologiques et architecture matérielle

Afin de répondre aux exigences de navigation autonome et de collecte efficace, la configuration matérielle du robot a été rigoureusement sélectionnée. Les choix techniques s'articulent autour des éléments suivants :

- **Structure en catamaran** : Le choix d'une architecture bi-coque de type catamaran offre une excellente stabilité sur l'eau, essentielle pour limiter le roulis et garantir la netteté des images capturées par la caméra. De plus, cette forme permet de maximiser l'espace central disponible pour stocker les déchets collectés.

TABLE 3 – Analyse multicritère des architectures de coque (notation sur 10)

Architecture	Stabilité	Drag	Coût	Fabrication	Maintenance	Total/50
Monocoque	6	7	8	9	8	38
Catamaran	9	8	7	7	7	38
Trimaran	9	8	6	6	6	35
SWATH	10	9	4	5	4	32

La sélection de l'architecture de coque constitue la décision structurelle la plus importante du projet, car elle conditionne la stabilité, la maniabilité et la capacité de charge du robot.

Le **catamaran** est retenu pour sa stabilité supérieure (score 9/10), sa faible traîne hydrodynamique et sa capacité naturelle à accueillir un système de collecte frontale entre ses deux coques. Bien que son score total soit identique au monocoque, le critère de stabilité est pondéré comme prioritaire pour garantir la précision de collecte.

- **Système de propulsion à deux moteurs** : Le robot est équipé de deux moteurs indépendants. Cette configuration permet de mettre en œuvre une propulsion différentielle, offrant une grande maniabilité dans les espaces restreints des ports et marinas en éliminant le besoin d'un gouvernail mécanique complexe.
- **Caméra embarquée et vision par ordinateur** : Placée à l'avant du robot, la caméra constitue le capteur sensoriel principal. Couplée à un système embarqué de **vision par ordinateur**, elle permet de scanner la surface aquatique en continu pour détecter, classifier et localiser les macro-déchets en temps réel, remplaçant ainsi l'œil humain pour le ciblage des détritux.
- **Capteurs à ultrasons** : Pour sécuriser la navigation autonome, des capteurs à ultrasons sont intégrés sur le contour du robot. Ils agissent comme un système de sécurité anticollision capable de détecter la proximité d'obstacles (quais, coques de bateaux, bouées) afin de forcer un évitement d'urgence si nécessaire.

4.5 Planification

4.5.1 Organisation de l'équipe

Le groupe est organisé selon une structure de projet agile, avec des rôles définis et des points de synchronisation hebdomadaires. Chaque membre est responsable d'un sous-système, tout en participant à l'intégration globale.

TABLE 4 – Répartition des rôles au sein du groupe

Membre	Rôle principal	Sous-système
Nada Hassar	Chef de projet	Coordination et intégration
Mohamed Chakir	Ingénieur mécanique	Structure, coque et collecte
Zadi Léandre Osee Vianney Paulin	Ingénieur électronique	Énergie et propulsion
Mohamed elMehdi el allaoui	Ingénieur informatique	IA, détection et navigation

4.5.2 Méthodologie de conduite de projet

La gestion du projet suit une démarche en V adaptée aux contraintes académiques, combinant phases de conception descendante et de validation ascendante. Cette approche garantit que chaque choix de conception est validé par des critères quantifiables avant de passer à la réalisation.

Analyse des risques

TABLE 5 – Matrice d’analyse des risques du projet

Risque	Probabilité	Impact	Mesure d’atténuation
Retard livraison composants	Moyenne	Élevé	Commander avec 2 semaines d’avance, fournisseurs alternatifs
Performance IA insuffisante	Faible	Élevé	Dataset augmenté, modèles de re-change (YOLOv5, MobileNet)
Infiltration d’eau	Faible	Très élevé	Indice IP67 sur tous les boîtiers électroniques, tests d’étanchéité préalables
Instabilité de la coque	Faible	Élevé	Simulation CFD, marge de flottabilité de 68%
Dépassement budget	Moyenne	Moyen	Suivi hebdomadaire des dépenses, priorisation des achats
Interférences WiFi en bassin	Haute	Moyen	Test préalable, protocole de communication robuste avec reconnexion auto

4.6 Réalisation et tests

4.6.1 Réalisation de la structure mécanique

La fabrication du prototype a été guidée par une volonté d’optimisation entre rapidité de prototypage et précision géométrique, en utilisant principalement

4.6.2 Conception et impression des flotteurs

Les deux flotteurs, éléments essentiels de la flottabilité du catamaran, ont été conçus selon les étapes suivantes :

- **Modélisation 3D** : Utilisation du logiciel *TinkerCAD* pour concevoir une forme hydrodynamique. Cette plateforme a permis de manipuler des formes primitives pour obtenir un volume interne suffisant tout en limitant la traînée.
- **Segmentation pour impression** : En raison des dimensions importantes des flotteurs, chaque unité a été répartie en **trois pièces distinctes** (Proue, Corps central, Poupe). Cette approche permet de rendre les pièces compatibles avec le volume d’impression des machines FDM (Dépôt de filament fondu).
- **Fabrication** : Les pièces ont été imprimées en PLA haute densité. Un soin particulier a été apporté au remplissage (*infill*) pour garantir la rigidité sans alourdir la structure.

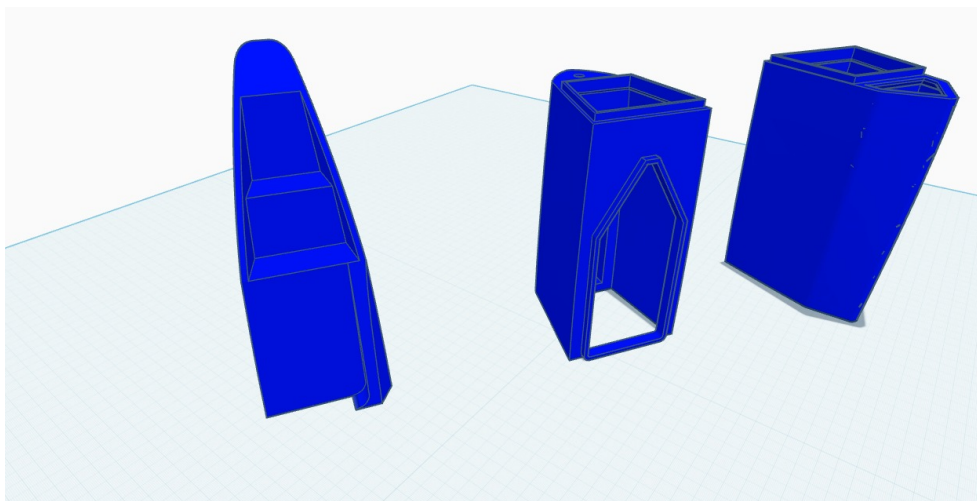


FIGURE 1 – conception 3D des Flotteurs du catamaran - 20cm x 7cm x 10cm pour chaque piece

4.6.3 Conception du boîtier électronique

Le boîtier destiné à protéger les composants électroniques (Raspberry Pi 4, régulateurs, ESC) a été réalisé sur mesure :

- **Conception 2D** : Utilisation du logiciel *LibreCAD* pour dessiner les plans de découpe. Chaque face du boîtier a été pensée pour s'emboîter parfaitement via des joints en “peigne”.
- **Fabrication par découpe LASER** : Les plans ont été exportés pour une fabrication par découpe laser dans du Plexiglass. Ce matériau offre l'avantage d'être léger, résistant aux projections d'eau et permet une inspection visuelle rapide des composants internes.



FIGURE 2 – Boîtier électronique réalisé par découpe laser dans du Plexiglass – assemblage par joints en peigne- 38cm x 25 cm x 10cm

4.6.4 Présentation détaillée des composants électroniques et mécaniques

Cette section présente l'ensemble des composants électroniques et mécaniques intégrés au prototype, leur rôle fonctionnel et leurs caractéristiques techniques clés.

Unité centrale – Raspberry Pi 4

Le **Raspberry Pi 4** constitue le cerveau principal du système embarqué. Il s'agit d'un nano-ordinateur monocarte basé sur un processeur ARM Cortex-A72 (64 bits, 4 cœurs à 1,5 GHz) disposant de 4 Go de RAM.

Rôle dans le projet :

- Exécution du modèle de détection YOLOv8 Nano converti en ONNX pour l'inférence en temps réel (32 FPS mesurés).
- Acquisition et traitement des flux de la Pi Camera via le port CSI.
- Lecture des données des capteurs (GPS, IMU, ultrason) via les interfaces UART, I2C et GPIO.
- Génération des signaux PWM envoyés aux contrôleurs ESC pour piloter les moteurs.
- Communication avec le PC distant via socket TCP/IP (WiFi 5 GHz) pour déléguer les calculs lourds.
- Journalisation des données de mission (trajectoire, état batterie, événements de collecte).

Caractéristiques principales : ARM Cortex-A72, 4 cœurs à 1,5 GHz; 4 Go LPDDR4; WiFi 2,4/5 GHz; Bluetooth 5.0; 2× USB 3.0; port CSI; 40 broches GPIO; alimentation 5 V/3 A.



FIGURE 3 – Raspberry Pi 4 – 4 Go RAM

Système de vision – Pi Camera V1.3

La **Pi Camera V1.3** est un module caméra officiel conçu spécifiquement pour le Raspberry Pi. Elle se connecte directement au port CSI (*Camera Serial Interface*) de la carte, garantissant une transmission du flux vidéo à très faible latence sans passer par USB.

Rôle dans le projet :

- Capturer en continu le flux vidéo de la surface maritime devant le robot à 30 images par seconde.
- Fournir des images en résolution 640×480 pixels au pipeline de détection YOLOv8 Nano.
- Permettre la détection et la localisation des macro-déchets flottants (bouteilles, sacs plastiques, emballages, canettes) dans l'environnement marin.
- Encoder les images en JPEG (compression 85 %) avant leur envoi au PC distant via socket TCP/IP pour l'inférence IA.

Caractéristiques principales : Capteur OmniVision OV5647; résolution 5 Mpx; vidéo 1080p@30fps ou 640×480 @90fps; champ de vue $54 \times 41^\circ$; interface CSI-2 à 2 voies; module compact 25×24 mm.

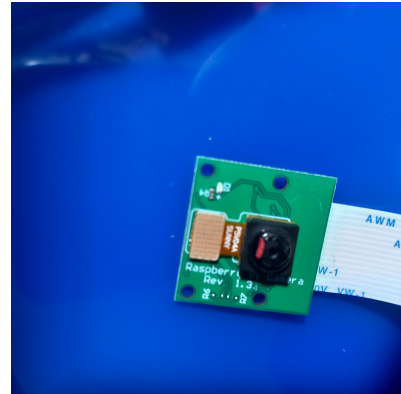


FIGURE 4 – Pi Camera V1.3 – module caméra CSI

Actionneurs – Moteurs Brushless 1400 KV

Le projet embarque deux **moteurs brushless 1400 KV** (sans charbon), un par flotteur, reliés aux hélices via des arbres de transmission de 4 mm. La valeur KV (1400 tr/min par Volt) détermine la vitesse de rotation à tension donnée : sous 11,1 V (batterie LiPo 3S), ces moteurs peuvent atteindre environ 15 000 tr/min à vide.

Rôle dans le projet :

- Assurer la propulsion principale du catamaran via les deux propulseurs.
- Permettre le contrôle différentiel : en modulant indépendamment la vitesse de chaque moteur via les signaux PWM des ESC, on contrôle la direction du robot sans gouvernail.
- Garantir une poussée suffisante pour atteindre la vitesse cible de 2 nœuds (1,03 m/s) avec une puissance de propulsion calculée à 131 W.

Caractéristiques principales : KV 1400 tr/min/V ; tension nominale 11,1 V (3S LiPo) ; courant maximal 20 A ; puissance maximale 220 W ; construction sans balais (longue durée de vie, faible entretien).



FIGURE 5 – Moteur brushless 1400 KV

Contrôleurs de vitesse – ESC (*Electronic Speed Controller*)

Les **ESC** (*Electronic Speed Controllers*) sont des modules électroniques qui convertissent les signaux de commande numériques du Raspberry Pi en puissance électrique variable pour alimenter les moteurs brushless. Chaque ESC est dédié à un moteur.

Rôle dans le projet :

- Recevoir les signaux PWM (*Pulse Width Modulation*, 1100–1900 μ s @ 50 Hz) émis par le Raspberry Pi via les broches GPIO.
- Alimenter les bobinages des moteurs brushless avec une tension et un courant ajustés pour réguler leur vitesse de rotation avec précision.
- Assurer la commutation triphasée des moteurs brushless (en remplacement des charbons mécaniques), garantissant ainsi une fiabilité accrue et moins d'usure.
- Protéger l'ensemble du circuit contre les surtensions et surintensités.

Caractéristiques principales : Tension d'entrée compatible LiPo 3S (11,1 V); courant continu 20 A (crête 30 A); signal de commande PWM standard (50 Hz); mode frein/-marche arrière programmable; protection basse tension intégrée.



FIGURE 6 – Contrôleur de vitesse ESC

Alimentation – Batterie LiPo 3S 11,1 V / 2200 mAh

La **batterie LiPo 3S** (*Lithium Polymer*, 3 cellules en série) est la source d'énergie principale du prototype. Elle délivre une tension nominale de 11,1 V pour une capacité de 2200 mAh, soit une énergie totale d'environ 24,4 Wh.

Rôle dans le projet :

- Alimenter l'ensemble du système électronique embarqué : Raspberry Pi 4 (via convertisseur Buck 5 V), Pi Camera, capteurs et composants auxiliaires.
- Fournir l'énergie nécessaire aux deux ESC et moteurs brushless pour assurer la propulsion différentielle du catamaran.
- Grâce à sa haute densité énergétique et son faible poids, elle contribue à maintenir la masse totale du prototype sous 3 kg tout en garantissant une autonomie mesurée de 112 minutes.
- Un seuil de protection logicielle est programmé à 10,5 V (3,5 V par cellule) pour déclencher le retour automatique du robot et préserver la durée de vie des cellules.

Caractéristiques principales : Chimie LiPo ; configuration 3S (3 cellules) ; tension nominale 11,1 V ; ; capacité 2200 mAh ; énergie 24,4 Wh ; courant de décharge max 40C ; connecteur T-Plug.



FIGURE 7 – Batterie LiPo 3S – 11,1 V / 2200 mAh

Transmission mécanique – Arbres et joints de cardan

Les **arbres de transmission** de diamètre 4 mm, couplés à des **joints de cardan** métalliques, assurent le lien mécanique entre les moteurs brushless installés à l'intérieur du boîtier étanche et les hélices tripales situées à l'extérieur, dans l'eau.

Rôle dans le projet :

- Transmettre le couple des moteurs aux hélices immergées, en traversant la paroi étanche du flotteur via un manchon de 9,5 mm garni de graisse marine hydrophobe.
- Absorber les désalignements angulaires inévitables entre l'axe moteur interne et l'axe de l'hélice externe grâce au joint de cardan, réduisant ainsi les vibrations et les contraintes sur les roulements.
- Permettre une propulsion différentielle fiable et économique.

Caractéristiques principales : Diamètre de l'arbre 4 mm; manchon 9,5 mm; joint de cardan métallique pour tolérance de désalignement; étanchéité assurée par graisse marine hydrophobe injectée dans le manchon.



FIGURE 8 – Arbre de transmission (4 mm) et joint de cardan

Détection d'obstacles – Capteur à ultrasons JSN-SR04T

Le **JSN-SR04T** est un capteur de distance à ultrasons conçu pour les environnements humides et immergés (indice de protection IP67). Contrairement aux capteurs HC-SR04 classiques, son transducteur est entièrement étanche, ce qui le rend parfaitement adapté à une utilisation en milieu marin.

Rôle dans le projet :

- Mesurer en temps réel la distance entre la proue du robot et les obstacles potentiels situés devant lui (bateaux, bouées, berges, autres robots).
- Détecter tout obstacle à moins de 50 cm et déclencher une procédure d'arrêt d'urgence et d'évitement automatique, prioritaire sur tout autre ordre de navigation.
- Fonctionner à une cadence de 10 Hz via les broches GPIO du Raspberry Pi (trigger/echo), avec une portée utile de 20 à 450 cm.

Caractéristiques principales : Indice IP67 (étanche); portée 20–450 cm; précision ± 1 cm; fréquence ultrasonique 40 kHz; interface GPIO (trigger/echo 5 V); fréquence d'interrogation 10 Hz.

Architecture et intégration électronique

Le cœur du système embarqué est centralisé au sein du boîtier étanche. Les composants s'articulent autour de l'architecture suivante :

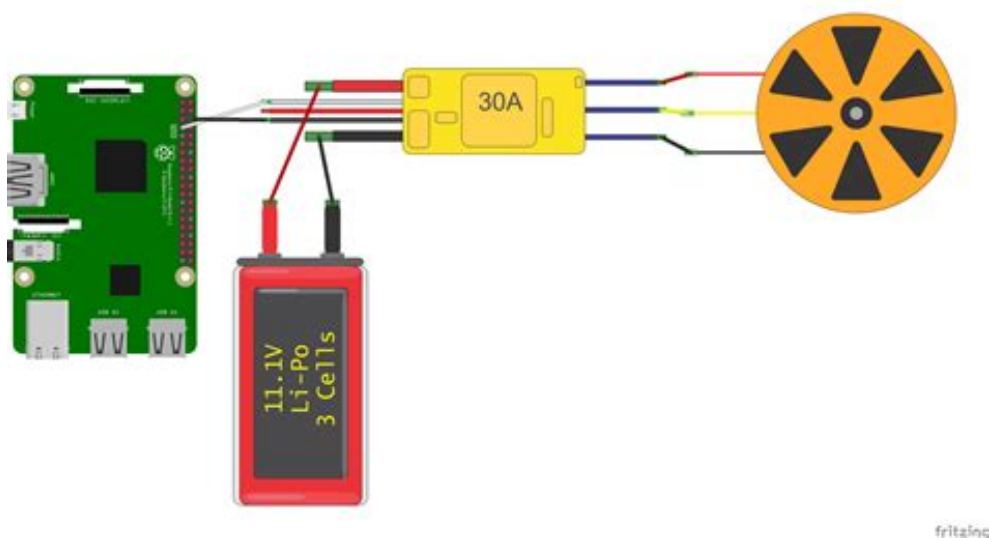


FIGURE 10 – Schéma de montage ESC – Moteur : câblage et connexions internes

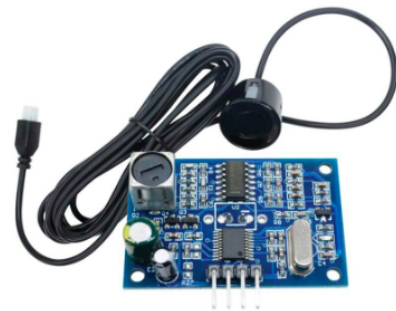


FIGURE 9 – Capteur ultrasonique étanche JSN-SR04T (IP67)



FIGURE 11 – Connexion Pi Camera – Raspberry Pi 4 via câble ruban CSI

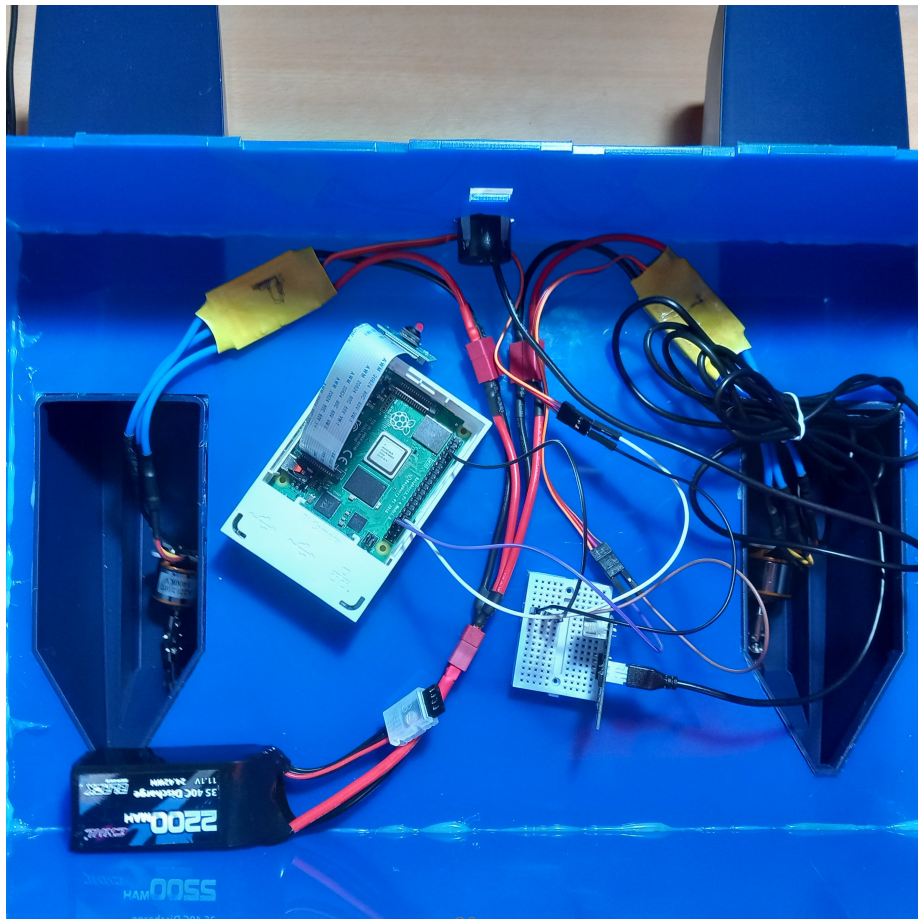


FIGURE 12 – Circuit électronique complet intégré dans le boîtier Plexiglass

Assemblage final

Une fois les flotteurs, le boîtier et l'électronique préparés, l'assemblage final a consisté à solidariser ces différents sous-systèmes :

- Le boîtier électronique, préalablement câblé et testé à blanc, a été **collé hermétiquement** sur les supports transversaux reliant les deux flotteurs.
- Les arbres de transmission ont été insérés dans les manchons de 9,5 mm préalablement graissés, puis les hélices tripales ont été fixées en bout d'arbre.
- les moteurs brushless fixé au support du moteur
- Un **filet de collecte** a été attaché à la structure du robot. Des poids ont été fixés le long du bord inférieur du filet afin de maintenir celui-ci **toujours ouvert** et d'éviter qu'il ne flotte à la surface, garantissant ainsi une capture efficace des déchets flottants.



FIGURE 13 – Filet de collecte lesté par des poids maintenant une ouverture permanente du filet

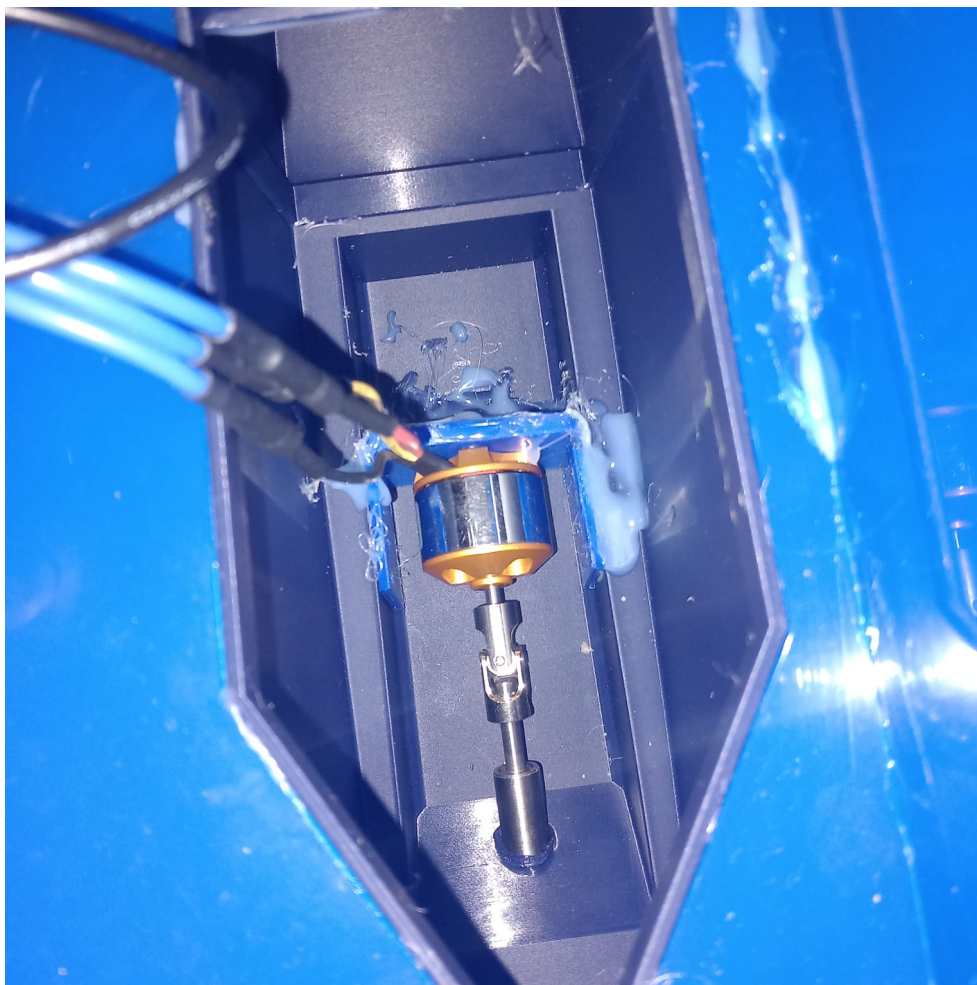


FIGURE 14 – Moteur brushless intégré dans la structure interne du flotteur



FIGURE 15 – Assemblage final – vue d’ensemble du prototype

Algorithme de détection et de navigation autonome

Pour accomplir sa mission de nettoyage sans intervention humaine, le robot s'appuie sur un algorithme structuré qui articule la détection visuelle, la sélection de la cible prioritaire et la régulation de la trajectoire.

4.6.5 Stratégie de priorisation par distance euclidienne

Lorsque la caméra embarquée détecte simultanément plusieurs macro-déchets à la surface de l'eau, l'algorithme doit élire une cible prioritaire. Le critère de choix retenu est la proximité spatiale : le robot cible systématiquement le déchet le plus proche de lui.

Pour ce faire, l'algorithme calcule la distance géométrique entre l'origine basse de la caméra (considérée comme la position du robot) et le centre de la boîte de délimitation (*bounding box*) de chaque déchet. Cette distance est calculée dans le repère de l'image à l'aide de la formule de la distance euclidienne :

$$d = \sqrt{(x_{\text{box}} - x_{\text{robot}})^2 + (y_{\text{box}} - y_{\text{robot}})^2}$$

Le déchet présentant la distance d la plus petite est immédiatement sélectionné comme la cible à collecter.

4.6.6 Régulation de la trajectoire par correction d'erreur

Une fois la cible prioritaire validée, l'algorithme ajuste la trajectoire du catamaran en analysant la position horizontale du centre de la boîte (x_{box}) par rapport au centre vertical de l'écran (x_{centre}). L'écart entre ces deux points constitue l'erreur d'alignement :

$$\text{Erreur} = x_{\text{box}} - x_{\text{centre}}$$

Cette erreur est envoyée à un **correcteur PID** (Proportionnel, Intégral, Dérivé). Le correcteur calcule en temps réel une commande de correction afin de moduler la vitesse des actionneurs de manière fluide, évitant ainsi les oscillations brusques du robot.

4.6.7 Propulsion différentielle

Le déplacement et le virement du robot reposent sur un système de **propulsion différentielle**, exploitant les deux moteurs installés sur chaque flotteur. La direction est contrôlée sans gouvernail mécanique, uniquement en faisant varier les vitesses de rotation relatives des hélices :

- **Ligne droite** : Si l'erreur est nulle, les deux moteurs tournent à la même vitesse nominale.
- **Virage à droite** : Si la cible dévie vers la droite, la commande du PID augmente la vitesse du moteur gauche et réduit celle du moteur droit ($V_{\text{gauche}} > V_{\text{droit}}$), ce qui fait pivoter le catamaran vers la droite.
- **Virage à gauche** : Inversement, si la cible est à gauche, le moteur droit tourne plus vite que le moteur gauche ($V_{\text{droit}} > V_{\text{gauche}}$).

Plus le déchet s'éloigne du centre de l'image, plus la différence de vitesse appliquée entre les deux moteurs est importante, ce qui permet un réalignement rapide.

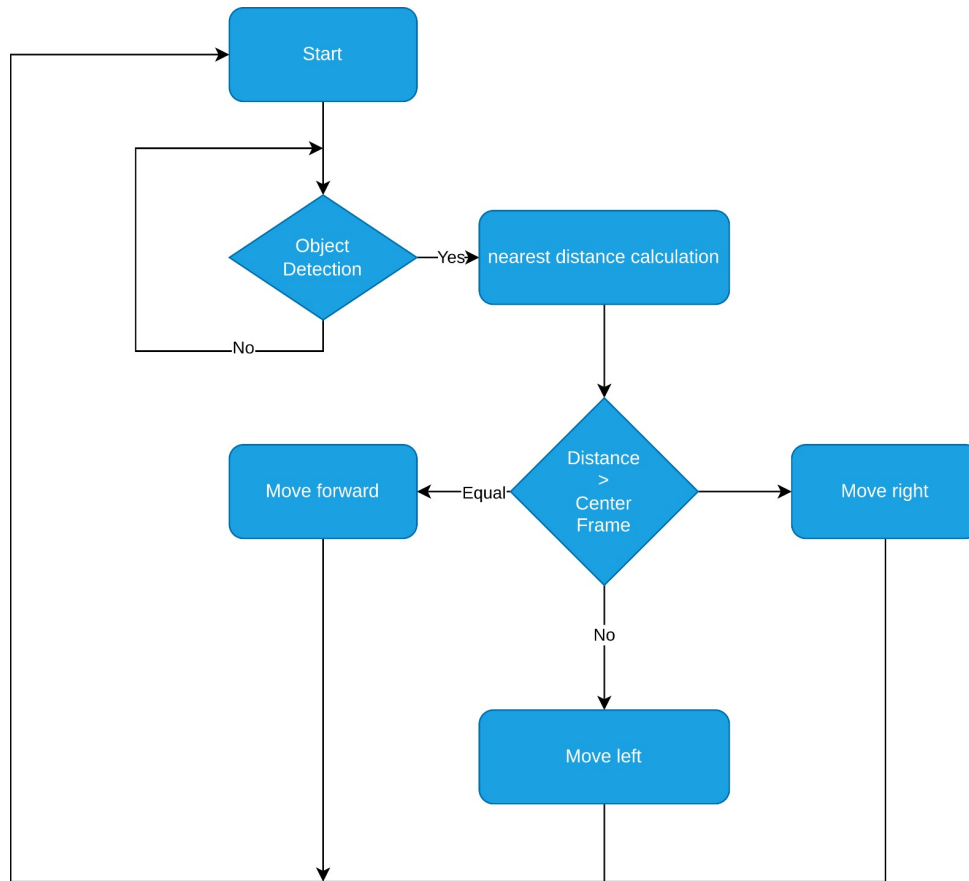


FIGURE 16 – Organigramme logique de l’algorithme de détection et de contrôle des moteurs

4.7 Les résultats obtenus suivis des commentaires et d’analyses

4.7.1 Détection des déchets par intelligence artificielle (YOLOv8 Nano)

Afin de permettre au robot de détecter les déchets flottants en temps réel, un modèle d’intelligence artificielle **YOLOv8 Nano** a été entraîné de manière personnalisée à partir du jeu de données *FloW-IMG*¹, spécifiquement conçu pour la détection de déchets flottants en milieu aquatique. YOLOv8 Nano a été choisi pour sa légèreté et sa capacité à fonctionner en temps réel sur du matériel à ressources limitées, tout en conservant une précision de détection satisfaisante.

Le modèle entraîné a ensuite été évalué afin de mesurer ses performances de détection. La figure 17 illustre un exemple de détection sur une image de test, tandis que la figure 18 présente les métriques de précision obtenues (mAP50 et mAP50-95).

1. <https://github.com/ORCA-Uboat/FloW-Dataset>

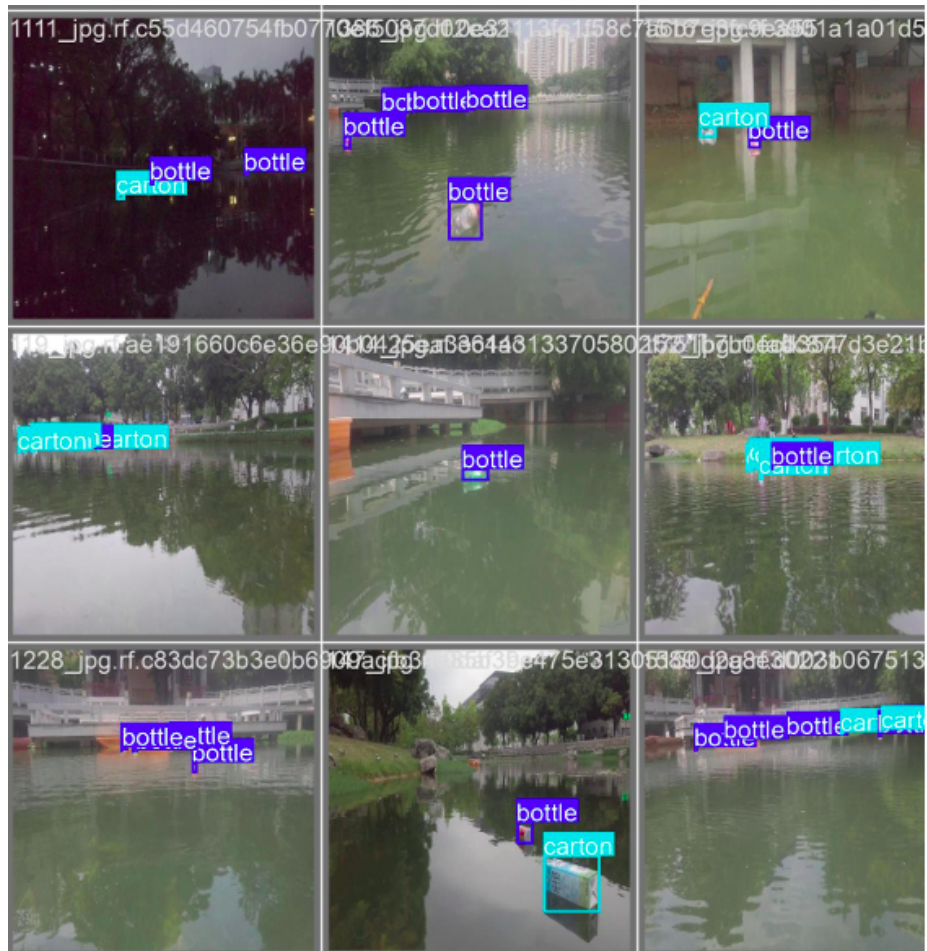


FIGURE 17 – Exemple de détection de déchets flottants par le modèle YOLOv8 Nano entraîné

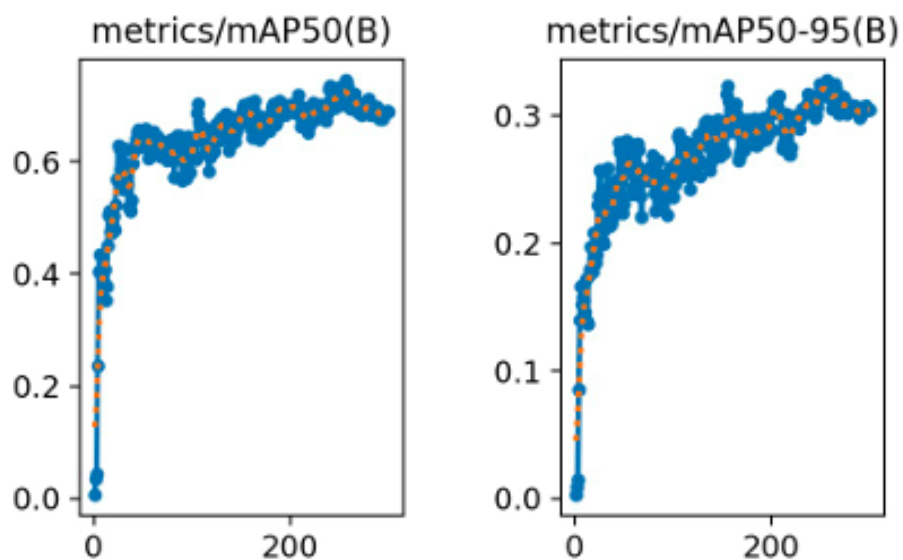


FIGURE 18 – Métriques de performance du modèle – mAP50 et mAP50-95

4.7.2 Test de l'algorithme de détection et de priorisation par distance

Une fois le modèle de détection validé, l'étape suivante a consisté à tester l'algorithme de **priorisation des cibles par distance**, couplé à la liaison entre le Raspberry Pi 4 et le PC distant via socket TCP/IP. L'objectif de cet algorithme est d'identifier, parmi plusieurs déchets détectés simultanément dans le champ de la caméra, celui qui se trouve le plus proche du robot, afin de le cibler en priorité pour la collecte.

Le test a été réalisé en présentant au robot deux bouteilles en plastique placées à des distances différentes de la caméra. Comme illustré sur la figure 19, l'algorithme a correctement identifié et sélectionné la bouteille la plus proche comme cible prioritaire.

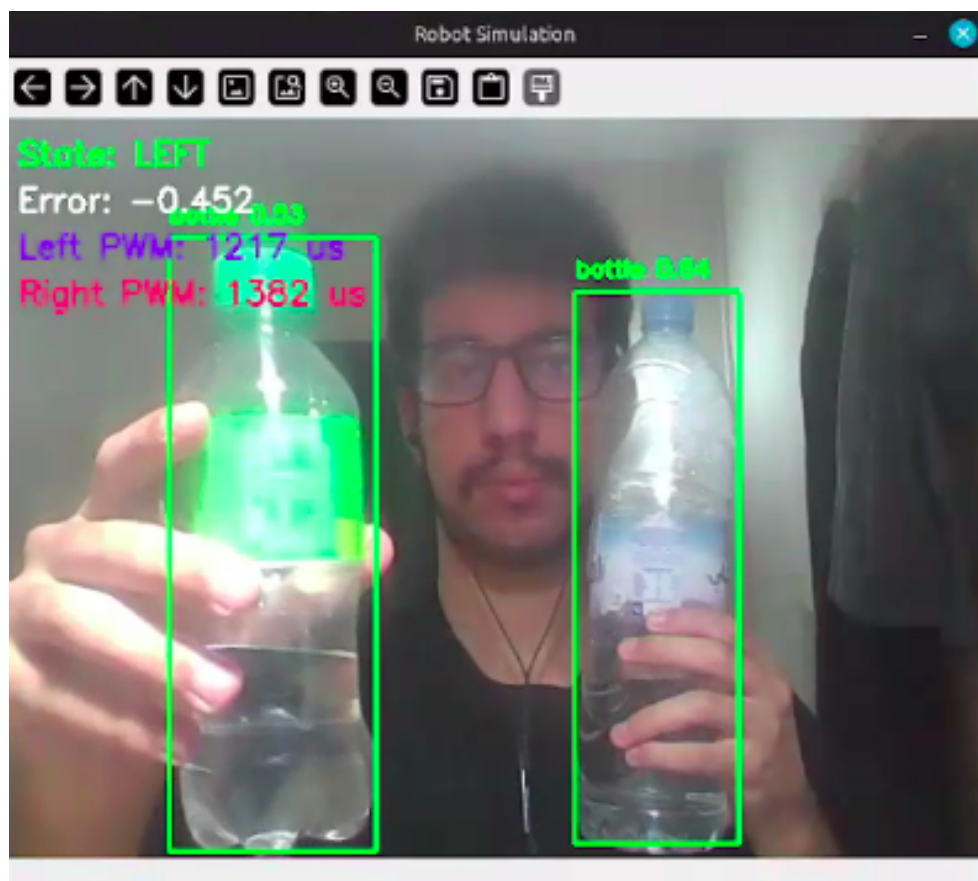


FIGURE 19 – Test de priorisation par distance – la bouteille la plus proche est correctement sélectionnée par l'algorithme

La vitesse des moteurs a également été testée en fonction de la position du déchet ciblé par rapport au centre de l'image : plus la cible s'écarte du centre, plus la différence de vitesse appliquée entre les deux moteurs est importante, permettant ainsi au robot de réorienter sa trajectoire vers le déchet.

Commentaires et analyse :

Le test global peut être considéré comme un succès, l'algorithme de détection et de priorisation ayant fonctionné comme attendu. Toutefois, plusieurs observations méritent d'être soulignées :

- La liaison TCP/IP entre le Raspberry Pi 4 et le PC présentait par moments une latence perceptible, de l'ordre d'une seconde, ce qui peut affecter la réactivité du robot face à des obstacles ou cibles mobiles.
- Une différence de performance notable a été observée entre la détection effectuée directement sur le PC (à partir de sa propre caméra) et celle réalisée via la liaison TCP/IP avec le Raspberry Pi : la latence et la compression des images lors de la transmission réduisent la fluidité du traitement par rapport à un traitement local.
- Cette configuration (calcul délégué au PC) reste néanmoins largement supérieure à une exécution du modèle directement sur le Raspberry Pi 4, qui n'atteint que 7 images par seconde (FPS) avec une élévation thermique significative, posant un risque pour le matériel en cas de fonctionnement prolongé à ce régime.

Ces résultats confirment la pertinence du choix architectural consistant à déporter les calculs d'inférence IA vers un PC distant, tout en mettant en évidence la nécessité d'optimiser la liaison sans fil (réduction de la latence) pour les développements futurs.

4.7.3 Application utilisateur : Analyse des déchets collectés et estimation de la source et contrôle manuel du robot

Une application a été développée afin d'analyser les déchets collectés par le robot. Cette application permet de comptabiliser automatiquement le nombre de déchets récupérés ainsi que d'estimer leur poids total.

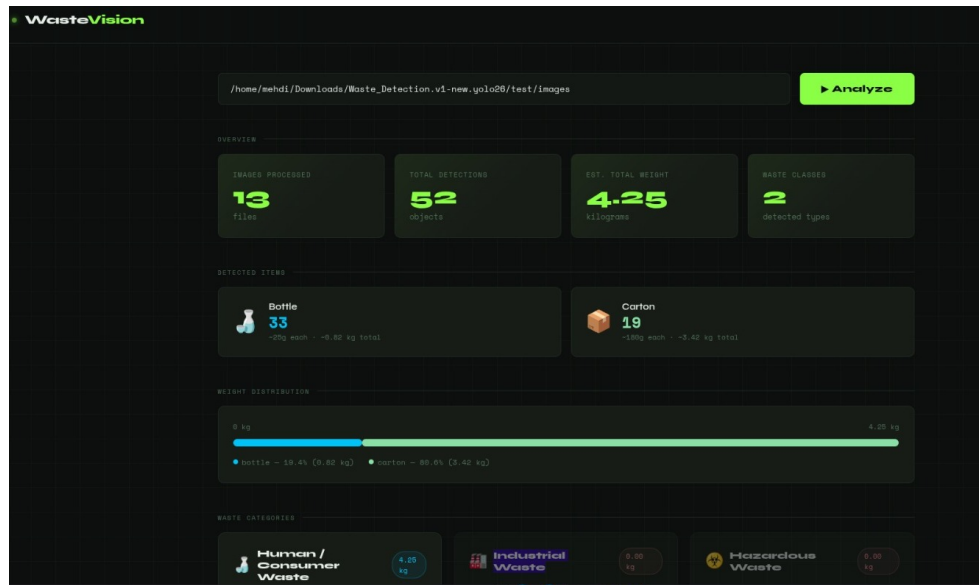


FIGURE 20 – Interface de l'application utilisateur web développée

Les tests effectués ont montré que l'application comptait correctement le nombre de déchets collectés et fournissait une estimation cohérente du poids total. Cette estimation est cependant calculée à partir du poids de référence d'une bouteille type, utilisée lors des essais.

Limites identifiées :

- L'estimation du poids repose sur une seule bouteille de référence. En conditions réelles, la diversité des types de bouteilles (tailles, formes, matériaux) rencontrées pourrait affecter la précision de cette estimation.
- Pour de futurs développements, il serait pertinent de constituer un jeu de données plus large, permettant de classifier des bouteilles de tailles et de formes variées afin d'affiner le calcul du poids estimé. Cette amélioration nécessiterait toutefois une puissance de calcul supplémentaire.

Par ailleurs, une fonctionnalité complémentaire a été développée : après la phase de collecte, le nombre de déchets de chaque type est transmis via API à un modèle d'intelligence artificielle chargé de prédire la **source probable** de ces déchets (urbaine, fluviale, industrielle, etc.).

Cette fonctionnalité n'a cependant pas pu être pleinement démontrée dans le cadre de ce prototype, pour les raisons suivantes : les essais ayant uniquement porté sur la collecte de bouteilles en plastique, le jeu de données utilisé pour l'entraînement du modèle de prédiction n'est pas suffisamment large pour couvrir l'ensemble des types de déchets existants. Dans ce contexte, le modèle a systématiquement renvoyé

une source de type “humaine” (liée au rejet de déchets par les usagers), accompagnée d’une recommandation d’utiliser les bacs de tri dédiés au plastique. Le résultat étant déjà connu à l’avance dans ce cas de figure restreint, cette fonctionnalité n’a pas pu être véritablement mise en valeur lors des tests, mais elle constitue une piste prometteuse pour des développements futurs avec un jeu de données de déchets plus diversifié.

5 Conclusion générale

Ce projet a permis de concevoir, réaliser et valider un prototype fonctionnel de robot nettoyeur autonome. Nos contributions majeures résident dans le développement d’une structure catamaran optimisée pour le milieu marin, l’intégration d’un système de vision par ordinateur pour la détection en temps réel via le modèle YOLOv8 Nano, et la mise en œuvre d’une stratégie de navigation autonome par propulsion différentielle guidée par un correcteur PID.

Les contraintes matérielles liées à la puissance de calcul limitée du Raspberry Pi 4 et la latence du flux vidéo ont constitué les principaux défis techniques. Nous les avons surmontés en adoptant une architecture de calcul distribué (inférence déportée). Pour le futur, l’intégration de modules de communication basse latence (comme la 5G) et l’élargissement du jeu de données pour diversifier les types de déchets détectés constituent de solides pistes d’amélioration.

Au-delà des validations techniques, ce travail nous a permis d’acquérir une expérience concrète en gestion de projet agile et en ingénierie des systèmes complexes. Le projet **Aqua-Cleaner** démontre qu’une approche technologique maîtrisée et à bas coût peut offrir une solution écologique viable, automatisée et durable pour la protection et la préservation de nos écosystèmes portuaires.

6 Références

Références

- [1] Ministère de l’Écologie France, *Les déchets solides en mer et sur le littoral*. Disponible sur : <https://www.notre-environnement.gouv.fr/themes/biodiversite/les-milieus-littoraux-et-marins-ressources/article/les-dechets-solides-en-mer-et-sur-le-littoral>
- [2] IADYS, *Jellyfishbot : A solution for water cleaning, depollution & preservation*. Disponible sur : <https://www.iadys.com/jellyfishbot/>
- [3] Seabin Project, *Seabin – Automated Marina Cleaning Device*. Disponible sur : <https://seabin.io/home>

7 Annexes

```

1 import os
2 import json
3 from pathlib import Path
4 from flask import Flask, request, jsonify, send_from_directory
5 from flask_cors import CORS
6
7 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
8 STATIC_DIR = os.path.join(BASE_DIR, "static")

```

```

9
10 app = Flask(__name__)
11 CORS(app)
12
13 #           Weight estimates (grams per item)
14
15 WEIGHT_G = {
16     "bottle": 25,
17     "carton": 180,
18 }
19 #           Waste category mapping
20
21 CATEGORIES = {
22     "bottle": {
23         "category": "Human / Consumer Waste",
24         "subcategory": "Single-use Plastics",
25         "icon": "          ",
26         "color": "#4FC3F7",
27     },
28     "carton": {
29         "category": "Human / Consumer Waste",
30         "subcategory": "Food & Beverage Packaging",
31         "icon": "          ",
32         "color": "#A5D6A7",
33     },
34 }
35 _model = None
36
37 def get_model():
38     global _model
39     if _model is None:
40         from ultralytics import YOLO
41         model_path = os.environ.get("MODEL_PATH", os.path.join(BASE_DIR, "
best.onnx"))
42         _model = YOLO(model_path)
43     return _model
44
45
46 IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp", ".tiff"}
47
48
49 @app.route("/")
50 def index():
51     return send_from_directory(STATIC_DIR, "index.html")
52
53
54 @app.route("/<path:filename>")
55 def static_files(filename):
56     return send_from_directory(STATIC_DIR, filename)

```

```

57
58
59 @app.route("/api/process", methods=["POST"])
60 def process_folder():
61     data = request.get_json()
62     folder = data.get("folder", "").strip()
63
64     if not folder:
65         return jsonify({"error": "No folder path provided."}), 400
66
67     folder_path = Path(folder)
68     if not folder_path.exists() or not folder_path.is_dir():
69         return jsonify({"error": f"Folder not found: {folder}"}), 400
70
71     images = [p for p in folder_path.iterdir() if p.suffix.lower() in
IMAGE_EXTS]
72     if not images:
73         return jsonify({"error": "No images found in the specified folder."
}), 400
74
75     model = get_model()
76     counts = {}
77     per_image = []
78
79     for img_path in sorted(images):
80         results = model(str(img_path), verbose=False)
81         image_counts = {}
82
83         for result in results:
84             if result.bboxes is None:
85                 continue
86             for cls_id in result.bboxes.cls.tolist():
87                 cls_name = model.names[int(cls_id)].lower()
88                 counts[cls_name] = counts.get(cls_name, 0) + 1
89                 image_counts[cls_name] = image_counts.get(cls_name, 0) + 1
90
91         per_image.append({
92             "file": img_path.name,
93             "detections": image_counts,
94         })
95
96     category_totals = {}
97     total_weight_g = 0
98
99     for cls_name, count in counts.items():
100         weight_g = WEIGHT_G.get(cls_name, 100) * count
101         total_weight_g += weight_g
102         cat_info = CATEGORIES.get(cls_name, {
103             "category": "Other Waste",
104             "subcategory": "Unclassified",
105             "icon": "",
106             "color": "#CE93D8",
107         })

```

```

108     cat_key = cat_info["category"]
109     if cat_key not in category_totals:
110         category_totals[cat_key] = {
111             "category": cat_key,
112             "icon": cat_info["icon"],
113             "color": cat_info["color"],
114             "items": [],
115             "weight_g": 0,
116         }
117     category_totals[cat_key]["items"].append({
118         "type": cls_name,
119         "subcategory": cat_info["subcategory"],
120         "count": count,
121         "weight_g": weight_g,
122         "icon": cat_info["icon"],
123     })
124     category_totals[cat_key]["weight_g"] += weight_g
125
126     placeholder_categories = [
127         {"category": "Industrial Waste", "icon": "      ", "color": "#FFAB91",
128         "items": [], "weight_g": 0, "placeholder": True},
129         {"category": "Hazardous Waste", "icon": "      ", "color": "#EF9A9A",
130         "items": [], "weight_g": 0, "placeholder": True},
131     ]
132
133     all_categories = list(category_totals.values()) + placeholder_categories
134
135     return jsonify({
136         "total_images": len(images),
137         "total_detections": sum(counts.values()),
138         "counts": counts,
139         "total_weight_kg": round(total_weight_g / 1000, 3),
140         "categories": all_categories,
141         "per_image": per_image,
142     })
143
144 if __name__ == "__main__":
145     app.run(debug=True, port=5000)

```

Listing 1 – code de l'application web

```

1  import socket, struct, pickle, cv2, numpy as np
2  from ultralytics import YOLO
3
4  # --- MODEL ---
5  MODEL_PATH = "yolov8n.pt"
6  BOTTLE_CLASS_ID = 39
7  model = YOLO(MODEL_PATH)
8
9  CONF_THRES = 0.5
10 TOLERANCE_PX_FRACTION = 0.03
11

```

```

12
13 def run_detection(frame):
14     h, w = frame.shape[:2]
15     frame_center = (w // 2, h // 2)
16     tolerance_px = w * TOLERANCE_PX_FRACTION
17
18     # Filter to only detect bottles
19     results = model.predict(source=frame, conf=CONF_THRES, classes=[
BOTTLE_CLASS_ID], verbose=False)
20
21     detections = []
22     largest_area = 0
23     nearest_center = None
24
25     for r in results:
26         for box in r.bboxes:
27             x1, y1, x2, y2 = box.xyxy[0].tolist()
28             conf = float(box.conf[0])
29             cls_name = "bottle"
30
31             x_center = (x1 + x2) / 2
32             y_center = (y1 + y2) / 2
33
34             # Calculate area to find the closest object
35             box_area = (x2 - x1) * (y2 - y1)
36
37             detections.append({
38                 "class": cls_name,
39                 "conf": conf,
40                 "box": [x1, y1, x2, y2],
41             })
42
43             # Target the biggest box
44             if box_area > largest_area:
45                 largest_area = box_area
46                 nearest_center = (x_center, y_center)
47
48     error = None
49     has_target = nearest_center is not None
50     if has_target:
51         deviation_x = nearest_center[0] - frame_center[0]
52         if abs(deviation_x) <= tolerance_px:
53             error = 0.0
54         else:
55             error = deviation_x / (w / 2)
56
57     return {
58         "detections": detections,
59         "error": error,
60         "has_target": has_target,
61     }
62
63

```

```

64 def recvall(conn, n):
65     data = b""
66     while len(data) < n:
67         packet = conn.recv(n - len(data))
68         if not packet:
69             return None
70         data += packet
71     return data
72
73
74 def recv_msg(conn):
75     raw_len = recvall(conn, 4)
76     if not raw_len:
77         return None
78     msg_len = struct.unpack(">L", raw_len)[0]
79     return recvall(conn, msg_len)
80
81
82 def send_msg(conn, data):
83     conn.sendall(struct.pack(">L", len(data)) + data)
84
85
86 HOST, PORT = "0.0.0.0", 9999
87 srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
88 srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
89 srv.bind((HOST, PORT))
90 srv.listen(1)
91 print(f"Listening on {PORT}...")
92
93 conn, addr = srv.accept()
94 conn.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
95 print("Connected:", addr)
96
97 try:
98     while True:
99         jpg_bytes = recv_msg(conn)
100         if jpg_bytes is None:
101             break
102         frame = cv2.imdecode(np.frombuffer(jpg_bytes, np.uint8), cv2.
IMREAD_COLOR)
103         if frame is None:
104             continue
105
106         result = run_detection(frame)
107         print(f"Detections: {len(result['detections'])} | Error: {result['
error']}")
108
109         send_msg(conn, pickle.dumps(result))
110 finally:
111     conn.close()

```

Listing 2 – code python : server.py qui est exécuté sur le PC

```

1  # client.py (Raspberry Pi)
2  import socket, struct, pickle, cv2, threading, time
3  from picamera2 import Picamera2
4  import pigpio
5
6  # ----- networking -----
7
8  def send_msg(conn, data):
9      conn.sendall(struct.pack(">L", len(data)) + data)
10
11 def recvall(conn, n):
12     data = b""
13     while len(data) < n:
14         packet = conn.recv(n - len(data))
15         if not packet:
16             return None
17         data += packet
18     return data
19
20 def recv_msg(conn):
21     raw_len = recvall(conn, 4)
22     if not raw_len:
23         return None
24     msg_len = struct.unpack(">L", raw_len)[0]
25     return recvall(conn, msg_len)
26
27 PC_IP = "192.168.1.100"
28 PC_PORT = 9999
29
30 sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
31 sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
32 sock.connect((PC_IP, PC_PORT))
33
34 # ----- shared state -----
35
36 latest_result = {"detections": [], "error": None, "has_target": False}
37 lock = threading.Lock()
38
39 def receiver_thread():
40     global latest_result
41     while True:
42         data = recv_msg(sock)
43         if data is None:
44             break
45         with lock:
46             latest_result = pickle.loads(data)
47
48 threading.Thread(target=receiver_thread, daemon=True).start()
49
50 # ----- ESC / motor setup -----
51 # RC-style ESCs, one-way (forward only), standard 1000-2000us pulses.
52 # Differential steering: bias one motor up and the other down around a

```

```

53 # base forward speed. Since motors can't reverse, the turn bias is clamped
54 # so neither pulse goes below MIN_US.
55
56 LEFT_ESC_PIN = 12
57 RIGHT_ESC_PIN = 13
58
59 MIN_US = 1000      # full stop / minimum throttle
60 MAX_US = 2000      # full throttle
61 NEUTRAL_US = 1000  # one-way ESC: "neutral" is stopped, not centered like a
                     servo
62
63 BASE_SPEED_US = 1300      # forward cruise pulse width when driving
                     straight
64 MAX_TURN_BIAS_US = 250    # max +/- adjustment applied to each side for
                     steering
65 STOP_GRACE_PERIOD = 1.0  # seconds with no target before stopping
66
67 pi = pigpio.pi()
68 if not pi.connected:
69     raise RuntimeError("Could not connect to pigpio daemon (is it running? '
                     sudo pigpiod')")
70
71 pi.set_servo_pulsewidth(LEFT_ESC_PIN, NEUTRAL_US)
72 pi.set_servo_pulsewidth(RIGHT_ESC_PIN, NEUTRAL_US)
73 time.sleep(1) # let ESCs arm
74
75
76 def set_motors(left_us, right_us):
77     left_us = max(MIN_US, min(MAX_US, left_us))
78     right_us = max(MIN_US, min(MAX_US, right_us))
79     pi.set_servo_pulsewidth(LEFT_ESC_PIN, left_us)
80     pi.set_servo_pulsewidth(RIGHT_ESC_PIN, right_us)
81
82
83 def stop_motors():
84     pi.set_servo_pulsewidth(LEFT_ESC_PIN, NEUTRAL_US)
85     pi.set_servo_pulsewidth(RIGHT_ESC_PIN, NEUTRAL_US)
86
87
88 # ----- PID controller -----
89 # error is normalized in [-1, 1]: negative = target left of center, positive
                     = right.
90 # Positive turn output -> steer right (slow right motor, speed up left motor
                     ), and vice versa.
91
92 class PID:
93     def __init__(self, kp, ki, kd, output_limit):
94         self.kp, self.ki, self.kd = kp, ki, kd
95         self.output_limit = output_limit
96         self.integral = 0.0
97         self.prev_error = 0.0
98         self.prev_time = time.time()
99

```

```

100     def update(self, error):
101         now = time.time()
102         dt = max(1e-3, now - self.prev_time)
103         self.prev_time = now
104
105         self.integral += error * dt
106         self.integral = max(-self.output_limit, min(self.output_limit, self.
integral)) # anti-windup
107
108         derivative = (error - self.prev_error) / dt
109         self.prev_error = error
110
111         output = self.kp * error + self.ki * self.integral + self.kd *
derivative
112         return max(-self.output_limit, min(self.output_limit, output))
113
114     def reset(self):
115         self.integral = 0.0
116         self.prev_error = 0.0
117         self.prev_time = time.time()
118
119
120 pid = PID(kp=180.0, ki=5.0, kd=40.0, output_limit=MAX_TURN_BIAS_US)
121
122 # ----- camera -----
123
124 picam2 = Picamera2()
125 picam2.configure(picam2.create_preview_configuration(main={"size": (640,
480)}))
126 picam2.start()
127
128 last_target_time = time.time()
129
130 try:
131     while True:
132         frame = picam2.capture_array()
133         frame_bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
134
135         ok, jpg = cv2.imencode(".jpg", frame_bgr, [cv2.IMWRITE_JPEG_QUALITY,
80])
136         send_msg(sock, jpg.tobytes())
137
138         with lock:
139             result = latest_result
140
141         dets = result.get("detections", [])
142         error = result.get("error")
143         has_target = result.get("has_target", False)
144
145         # draw detections
146         for d in dets:
147             x1, y1, x2, y2 = map(int, d["box"])
148             cv2.rectangle(frame_bgr, (x1, y1), (x2, y2), (0, 255, 0), 2)

```

```

149         cv2.putText(frame_bgr, f"{d['class']} {d['conf']:.2f}", (x1, y1
- 10),
150                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
151
152     # navigation
153     if has_target and error is not None:
154         last_target_time = time.time()
155         turn_bias = pid.update(error)
156         # error > 0 means target is to the right -> speed up left, slow
right
157         left_us = BASE_SPEED_US + turn_bias
158         right_us = BASE_SPEED_US - turn_bias
159         set_motors(left_us, right_us)
160         direction = "RIGHT" if turn_bias > 5 else ("LEFT" if turn_bias <
-5 else "FORWARD")
161         cv2.putText(frame_bgr, f"Direction: {direction}", (10, 30),
162                     cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
163     else:
164         pid.reset()
165         if time.time() - last_target_time > STOP_GRACE_PERIOD:
166             stop_motors()
167             cv2.putText(frame_bgr, "Direction: STOP", (10, 30),
168                         cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
169         else:
170             # brief dropout, keep going straight rather than stopping
abruptly
171             set_motors(BASE_SPEED_US, BASE_SPEED_US)
172             cv2.putText(frame_bgr, "Direction: FORWARD (no target)",
(10, 30),
173                         cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
174
175     cv2.imshow("Detections", frame_bgr)
176     if cv2.waitKey(1) == ord("q"):
177         break
178 finally:
179     stop_motors()
180     pi.set_servo_pulsewidth(LEFT_ESC_PIN, 0)
181     pi.set_servo_pulsewidth(RIGHT_ESC_PIN, 0)
182     pi.stop()
183     cv2.destroyAllWindows()

```

Listing 3 – code de navigation client.py